zensar

ZENSAR CONTENT STUDIO

User Manual

This manual is for anyone, including people who have never seen the project. Part 1 gets the app running, Part 2 explains how to use it, and Part 3 says what every file in the project is for, in plain words.

Offline AI · Your documents never leave the machine

Contents

What is Zensar Content Studio?

Part 1 — Getting started

1.1 What you need

1.2 Get the project onto your computer

1.3 Open the project folder

1.4 Start the app with start.bat

1.5 Stop the app

1.6 Start it again later

1.7 Special cases

Part 2 — Using the app

2.1 Dashboard

2.2 New project → turn documents into slides, visuals and a video

2.3 Projects

2.4 Research Papers → reformat a paper for a publisher

2.5 Brand Documents → make a branded Zensar white paper

2.6 Templates

2.7 Media Library

2.8 Brand Manager

2.9 Settings

Part 3 — Every file explained

3.1 Files in the main folder

3.2 docs/ — documentation (this folder)

3.3 scripts/ — helper programs

3.4 backend/ — the engine (Python)

3.5 frontend/ — the screens you see (React)

3.6 brands/zensar/ — the Zensar brand

3.7 samples/ — examples to try

3.8 tests/ — automatic checks

3.9 Folders created while the app runs (on your computer only)

Frequently asked questions

This manual is for anyone, including people who have never seen the project. Part 1 gets the app running, Part 2 explains how to use it, and Part 3 says what every file in the project is for, in plain words.

What is Zensar Content Studio?

Zensar Content Studio is a program that runs **on your own computer** and turns documents into finished material:

You give it	You get
A report, Word file, PDF, demo video or code repository	A **PowerPoint presentation**, brand-styled **images and diagrams**, and a **narrated video**
A research paper	The same paper re-formatted for a publisher: **IEEE, Springer, Elsevier, ACM or APA** (Word, PDF and LaTeX)
Any document (PDF, Word, text)	A **branded Zensar white paper** (PDF) designed like a reference document, with pictures created for its content

Everything happens on your computer. Your documents are never uploaded to the internet, and the AI models run locally.

Part 1 — Getting started

1.1 What you need

Item	Notes
A Windows 10 / 11 PC, or Windows Server 2019 / 2022	16 GB of memory recommended
About 15 GB of free disk space	For the programs and AI models
Internet on the **first** start only	Afterwards the app works fully offline
(Optional) An NVIDIA graphics card	Makes image generation about 10× faster

You do **not** need to install anything by hand. `start.bat` installs everything that is missing.

1.2 Get the project onto your computer

Option A: download as a ZIP (easiest)

1. Open **https://github.com/V-JasonEmmanuel/Zen-TMT** in your browser.
2. Click the green **Code** button, then **Download ZIP**.
3. Open your **Downloads** folder, right-click **Zen-TMT-main.zip** and choose **Extract All...**. Pick a short location such as `D:\Zen-TMT`, then click **Extract**.

Option B: with Git

1. Press the **Windows key**, type `cmd` and press **Enter**.
2. Type these commands, pressing **Enter** after each: `D: git clone https://github.com/V-JasonEmmanuel/Zen-TMT.git` This creates the folder `D:\Zen-TMT`.

1.3 Open the project folder

1. Press **Windows key + E** to open **File Explorer**.
2. Go to the folder you extracted or cloned, for example `D:\Zen-TMT`.
3. You should see folders such as `backend`, `frontend`, `docs` and `scripts`, and files such as **`start.bat`** and **`stop.bat`**.

> If you see only one folder called `Zen-TMT-main`, open it. `start.bat` is inside.

1.4 Start the app with `start.bat`

1. **Double-click `start.bat`.**
   - If Windows shows "*Windows protected your PC*", click **More info → Run anyway**. This appears only because the file was downloaded.

2. A black window titled **"Zensar Content Studio – setup"** opens and works through numbered steps. The **first time**, this takes about 10–30 minutes. Later starts take seconds.

Step shown	What it does (in plain words)
`[0] Microsoft Visual C++ runtime`	Installs a small Windows component that the AI parts need (only if it is missing).
`[1] Python`	Finds or installs Python, the language the app is written in, and creates a private copy for the app (`.venv` folder).
`[2] Python packages`	Downloads the building blocks the app uses (about 300 MB).
`[3] Configuration`	Creates the settings file `.env` .
`[4] Document understanding model`	Downloads a small AI model that understands what text means (about 90 MB).
`[5] Web interface`	Prepares the screens you will see in the browser.
`[6] Image generation`	Installs the picture-making AI (uses your graphics card if you have one) and its model (about 2 GB). It **asks first**; type **Y** and press **Enter**.
`[7] Local AI`	Installs **Ollama** (the local writing AI) and a language model. It **asks first**; type **Y** and press **Enter**.
`[8] System check`	Checks that everything needed is present.
`[9] Starting Content Studio`	Starts the app.

3. When you see **"Content Studio is running at http://localhost:8000"**, your web browser opens the app automatically.
    - If it doesn't open, open any browser and type **http://localhost:8000** in the address bar.
4. A small minimised window called **"Zensar Content Studio (close to stop)"** keeps the app running. **Leave it open** while you work.

Tip: install without questions. Open `cmd` in the project folder and run `set ZCS_ASSUME_YES=1 && start.bat` . Every question is answered "yes".

1.5 Stop the app

- Close the **"Zensar Content Studio (close to stop)"** window, **or**
- Double-click `stop.bat` in the project folder.

1.6 Start it again later

Double-click `start.bat` again. It skips everything that is already installed and opens the app in a few seconds.

1.7 Special cases

Situation	What to do
Another program already uses port 8000	Nothing. `start.bat` picks the next free port (8001, 8002, ...) and tells you the address.
Company network blocks pypi.org (corporate proxy)	Nothing. `start.bat` then downloads everything it needs from GitHub (the Zen-TMT-offline bundle). This needs Python 3.12.
Server in the cloud	Use Remote Desktop and open http://localhost:8000 on the server. To open it from your own PC, see "Deploying on a Windows cloud GPU server" in `README.md`.
Setup failed	Read the red message. Details are in the `logs` folder (`pip.log` , `npm.log` , `server.log`). Fix the cause (usually the internet connection) and run `start.bat` again. It continues where it stopped.

Part 2 — Using the app

The left-hand menu has these sections:

2.1 Dashboard

The home screen. It shows your recent projects and whether the local AI, models and tools are ready (green dots). Click **Settings** if something shows as missing.

2.2 New project → turn documents into slides, visuals and a video

1. Click **New project**.
2. **Sources:** drag your files into the box. You can add documents (PDF, Word, PowerPoint, text), demo videos (.mp4) or a zipped code repository. You can also paste a GitHub address.
3. **Brand & template:** choose a design, for example Corporate, Bold or Minimal.
4. **Instruction:** describe what you want, for example "*10-slide executive summary for the leadership team*".
5. **Outputs & video:** choose the narration voice, music and any extras.
6. Click **Generate**. A progress bar shows each stage.
7. When it finishes, the project opens with these tabs:
   - **Presentation:** view, present full-screen, download PPTX/PDF, edit or regenerate any slide.
   - **Images:** the generated diagrams and charts.
   - **Video editor:** arrange clips, trim, change narration, render the MP4.
   - **Sources:** which page each statement came from.
   - **Content plan / Files:** the plan and every output file.

2.3 Projects

A list of all projects. Click one to reopen it. You can rename, delete or regenerate it.

2.4 Research Papers → reformat a paper for a publisher

1. Click **Research Papers**.
2. **Your paper:** drop a PDF, Word file, LaTeX file or ZIP, or paste the text.
3. **Target format:** click one, for example *Springer Nature journal* or *IEEE*.
4. Click **Convert paper**. Wait until the badge says **Converted**.
5. Look through the tabs:
   - **Preview:** the paper in the new layout.
   - **Structure:** title, authors and sections. You can correct anything here.
   - **References:** every reference, parsed and re-formatted. Click one to fix it.
   - **Report:** what the publisher requires and how much text was preserved.
   - **Downloads:** Word, PDF, LaTeX project (for Overleaf) and BibTeX.

6. To try another publisher, choose a new format. The paper is not read again.

2.5 Brand Documents → make a branded Zensar white paper

1. Click **Brand Documents**.
2. **Brand template:** click a template card, for example *Zensar White Paper*.
   - New reference? Under **Add a reference PDF**, drop a branded PDF, give it a name and click **Learn template**. The app studies its design.
   - **Details** shows the colours, fonts and back-cover text it learned. You can edit them.
3. **Your document:** drop a PDF, Word or text file, or paste text.
4. **Length:**
   - **Match the reference** (recommended): about the same number of words and pages as the reference. Long documents are shortened and short ones lengthened by the local AI, keeping all key facts.
   - **Custom length:** choose a word count with the slider.
   - **Full document:** every word, as many pages as needed.
5. **Options:** title, cover label, number of image pages, and whether to generate images.
6. Click **Create branded PDF**. The progress bar shows each step: reading, fitting the length, creating images and laying out pages.
7. Results: a PDF preview, page thumbnails, the generated images (with the description used for each), and a check that the text arrived. Click **Download PDF**.
   - **New images** creates different pictures.
   - **Create again** applies a different title, label or template.

2.6 Templates

Ready-made presentation designs, built only from the brand's own colours. You can also upload your own PowerPoint template.

2.7 Media Library

All images, videos and audio the app can use: brand assets, your uploads and generated visuals, each with its licence.

2.8 Brand Manager

The brand's colours, fonts, logos and rules, learned from official references. You can review and edit them.

2.9 Settings

Choose the local AI model and voice, and see the privacy settings. **Offline mode** is on by default.

Part 3 — Every file explained

The project folder contains these folders and files. Folders created on your computer while the app runs (`data` , `outputs` , `projects` , `models` , `logs` , `.venv` , `offline`) are **not** part of the download.

3.1 Files in the main folder

File	What it is / what it does
`start.bat`	**The one file you double-click.** Installs whatever is missing, then starts the app and opens your browser.
`stop.bat`	Stops the running app.
`README.md`	The technical introduction to the project (for developers).
`.env.example`	A sample settings file. `start.bat` copies it to `.env` the first time.
`requirements.txt`	The list of Python building blocks the app needs. `start.bat` installs them.
`requirements-images.txt`	Extra building blocks for creating pictures (image AI).
`requirements-prepare.txt`	Tools needed only once to prepare the text-understanding model (developers).
`pytest.ini`	Settings for the automatic tests.
`.gitignore`	Tells Git which files must never be uploaded (your documents, models, logs).
`.gitattributes`	Tells Git how to treat line endings in files.

3.2 `docs/` — documentation (this folder)

File	What it is
`User_Manual.pdf`	This manual.
`Project_Documentation.pdf`	What the project is, how it is built and the technology used.
`Pipeline_Explanation.pdf`	How each feature works, step by step.
`Module_Architecture.pptx`	Slides explaining the parts (modules) of the app.
`Pipeline.pptx`	Slides explaining the processing pipelines.
`demo/Zensar_Content_Studio_Demo.mp4`	The demo video.
`source/`	The text of the three PDFs, for editing. After a change, run `scripts/make_docs_pdfs.py` to rebuild the PDFs.

3.3 `scripts/` — helper programs

File	What it does
`bootstrap.ps1`	The real installer behind `start.bat`. Checks and installs Python, packages, models, the interface and the AI, then starts the app.
`setup.bat`	An older setup shortcut. It runs the same installation as `start.bat`.
`check_environment.py`	Checks the computer and lists what is ready and what is missing (step 8 of start-up).
`fetch_models.py`	Downloads the AI models: text understanding, and with `--images` the picture model.
`prepare_models.py`	Developer tool: converts the text-understanding model into a faster format.
`offline_bundle.py`	Packs everything into pieces for computers that can only reach GitHub, and puts the pieces back together on the server.
`make_offline_bundle.bat`	Builds a folder of packages on a PC with internet, to copy to an offline machine.
`record_demo.py`	Records the demo video by clicking through the app automatically.
`edit_demo.py`	Cuts and speeds up the recording into the final demo video.
`make_docs_pdfs.py`	Turns the documentation text in `docs/source/` into the PDF documents.
`zensar_brand_research.py`	One-time tool that collected the Zensar brand facts (colours, logos) from official Zensar sources.
`smoke_pipeline.py`	Developer check: runs reading, understanding and planning on one file.
`smoke_render.py`	Developer check: makes slides and images from a test document.
`e2e_media_test.py`	Developer check of the media features against a running app.

3.4 `backend/` — the engine (Python)

The backend does all the work: reading files, the AI, making slides, papers, PDFs and videos.

File	What it does
`main.py`	**Starts the server.** Connects all the parts and serves the screens to your browser.
`schemas.py`	The shared "forms" every step fills in, for example what a slide or a content plan looks like.

`backend/api/` — the doors the screens knock on

Each file receives requests from one screen and passes them to the right part of the engine.

File	Handles
`documents.py`	Uploading documents, demo videos, code archives and GitHub links.
`projects.py`	Projects: list, open, edit the plan, regenerate a slide, download files.

File	Handles
`generation.py`	Starting generation jobs and reporting progress.
`brands.py`	The Brand Manager: brand profiles, assets and reference analysis.
`templates.py`	Analysing presentation templates.
`media.py`	The Media Library.
`video.py`	The video editor: timeline, script import and rendering.
`papers.py`	Research Papers: convert, review, correct and download.
`branddocs.py`	Brand Documents: learn templates, create branded PDFs, download them.
`settings.py`	Settings and the system status shown on the Dashboard.

`backend/ingestion/` — reading files

Each file knows how to read one kind of file and turn it into clean text blocks with page numbers.

File	Reads
`base.py`	The common rules every reader follows (a "plug-in" system for new file types).
`pdf.py`	PDF files (uses OCR only for scanned pages).
`docx.py`	Word documents.
`pptx.py`	PowerPoint files (each slide becomes a page).
`html.py`	Web pages (scripts are ignored, nothing is run).
`markdown.py`	Markdown text files.
`txt.py`	Plain text files (guesses the headings).
`video.py`	Demo videos: finds the scenes, describes each one and writes a narration.
`repo.py`	Code repositories: README, technology, components, API, architecture.

`backend/extraction/` — understanding the layout

File	What it does
`structure.py`	Finds sections, their types (summary, method, results…) and the title.
`ocr.py`	Reads text from scanned pages (optional, offline).

`backend/intelligence/` — finding the right facts

File	What it does
`chunking.py`	Cuts documents into small passages, remembering which page each came from.

File	What it does
`embeddings.py`	Turns each passage into numbers that capture its meaning (local AI model).
`retrieval.py`	Finds the passages most similar in meaning to a question (vector search).
`relevance.py`	Ranks passages by how relevant they are to your instruction and drops the rest.
`verification.py`	Checks every generated statement against its source. Numbers must match exactly.
`text.py`	Small text helpers used by the parts above.

`backend/llm/` — talking to the local writing AI

File	What it does
`__init__.py`	Gives the rest of the app the configured local AI.
`base.py`	The common interface, so any local AI model can be plugged in.
`ollama.py`	Talks to Ollama on this computer only (refuses outside servers).
`prompts.py`	Loads the AI instruction templates from the `prompts` folder.
`structured_output.py`	Never trusts raw AI output: extracts it, validates it, repairs it or asks again.
`prompts/*.txt`	The instructions given to the AI: system rules, slide writer, visual planner, narration writer, instruction parser.

`backend/planning/` — deciding what goes where

File	What it does
`content_contract.py`	Turns your instruction into a clear specification (how many slides, audience, focus).
`content_planner.py`	Builds the slide-by-slide plan from the relevant, verified facts.
`slide_planner.py`	Rules for each slide layout (what fits, safe fall-backs).
`visual_planner.py`	Decides which slides get a chart, diagram or picture (charts only from real table numbers).
`video_planner.py`	Decides how long each scene lasts in the video.
`script.py`	Reads a narration script you supply and splits it per slide.

`backend/pipeline/` — running the jobs

File	What it does
`jobs.py`	Runs jobs one at a time in the background and records progress.
`orchestrator.py`	The step-by-step recipe for each job: read, understand, plan, verify, render.
`sources.py`	Combines several sources (documents, videos, repositories) in one project.

File	What it does
`outputs.py`	Writes every result file: PowerPoint, PDF, images and video.
`video.py`	Builds and renders the project video.

`backend/rendering/` — drawing slides, images and videos

File	What it does
`scene.py`	A drawing description that all three output types (PowerPoint, image, SVG) can draw identically.
`layouts.py`	Decides where everything goes on a slide, using the brand's sizes and colours.
`components.py`	Re-usable building blocks: title bars, bullet lists, KPI tiles, diagrams.
`grid.py`	The invisible grid that keeps everything aligned.
`motifs.py`	Zensar's shape language (circle, square, triangle) for decorations.
`experience.py`	Zensar "Experience" designs: cover, divider and gradient styles.
`charts.py`	Brand-styled charts.
`images/raster.py`	Draws scenes as PNG/JPG pictures (previews, PDF, video frames).
`images/svg.py`	Draws scenes as SVG vector images.
`ppt/generator.py`	Builds the editable PowerPoint file.
`video/timeline.py`	The editable list of video clips (trim, crop, speed, narration).
`video/composer.py`	Turns the timeline into an MP4 with motion, narration, music and subtitles.
`video/motion.py`	The animations: titles fly in, diagrams build, numbers count up.
`video/narration.py`	Turns text into speech with offline Windows voices (or Piper).
`video/audio.py`	Mixes narration and music and lowers the music under speech.
`video/storyboard.py`	Simple video assembly from slide pictures and narration.
`video/ffmpeg.py`	Finds and runs FFmpeg, the video encoding tool.

`backend/branding/` — the brand rules

File	What it does
`brand_profile.py`	Loads and saves brand profiles (colours, fonts, logos). Never invents a value.
`theme.py`	Turns a brand profile into exact colours, fonts and sizes, with neutral fall-backs when a value is missing.
`fonts.py`	Finds the fonts installed on the computer and measures text.

File	What it does
`presets.py`	Presentation designs (Corporate, Bold, …) built from brand colours.
`template_parser.py`	Reads a PowerPoint template and suggests brand values from it.
`reference_analyzer.py`	Reads brand pictures and suggests colours, which you then confirm.
`social.py`	The Zensar "Experience" design language measured from Zensar's LinkedIn posts.

`backend/media/` — pictures and media

File	What it does
`library.py`	The Media Library storage. Each file is stored once, with its details and licence.
`treatment.py`	Simple picture edits: crop, brightness, brand tint, borders.
`online.py`	**Optional** licensed image search (off by default; sends only generic words).
`query.py`	Builds safe search words that never contain document text.

`backend/vision/` — understanding videos

File	What it does
`keyframes.py`	Finds where the scenes of a screen recording change.
`describe.py`	Describes each scene picture with a local vision AI.

`backend/services/`

File	What it does
`documents.py`	Stores an uploaded document and reads it once, keeping the results.
`github.py`	Downloads a GitHub repository as a ZIP. This is the only feature that goes online, and only when you ask.

`backend/storage/`

File	What it does
`database.py`	Saves and loads projects, jobs and settings in a small local database (SQLite).
`models.py`	The tables of that database.

`backend/utils/` — helpers

File	What it does
`config.py`	Reads the settings from `.env`: folders, port, AI model.

File	What it does
`files.py`	Safe file names and paths.
`logging.py`	Writes the log. It records counts and timings, **never** document text.
`network_guard.py`	**Privacy guard:** blocks every connection to the internet except this computer.
`progress.py`	Reports progress of long steps to the screen.

`backend/papers/` — Research Papers

File	What it does
`service.py`	Runs a conversion from start to finish and stores the results.
`model.py`	The description of a paper (title, authors, sections, references) shared by every reader and writer.
`formats.py`	The rules of each publisher (IEEE, Springer, Elsevier, ACM, APA): page size, columns, fonts, numbering.
`structure.py`	Works out the title, authors, abstract and sections from what the readers found.
`references.py`	Splits the reference list and reads each entry into fields (authors, year, title, journal…).
`citations.py`	Finds citations like [3] or (Smith, 2020) in the text and links them to references.
`crossrefs.py`	Links "Table 2", "Fig. 3", "Section 2.1" so they renumber in the new format.
`cite_render.py`	Formats citations and references in the publisher's official style.
`math_render.py`	Draws LaTeX formulas as pictures for Word and PDF.
`checks.py`	The report: the publisher's rules and how much of the text was kept.
`readers/pdf.py`	Reads PDF papers: columns, headings, broken words, formulas, tables, figures.
`readers/docx.py`	Reads Word papers, including Word equations.
`readers/latex.py`	Reads LaTeX papers (.tex or a .zip project).
`readers/omml.py`	Converts Word equations into LaTeX.
`readers/text.py`	Reads Markdown, plain text and pasted text.
`writers/tex.py`	Writes the LaTeX project in the publisher's official class.
`writers/word.py`	Writes the Word file in the publisher's layout.
`writers/pdf.py`	Writes the PDF preview in the publisher's layout.
`writers/bibtex.py`	Writes the references as a BibTeX file.
`writers/common.py`	Rules shared by all writers: numbering, captions, statements.

File	What it does
`styles/*.csl`, `styles/NOTICE.md`	The publishers' official citation style files and their licence notice.

`backend/branddocs/` — Brand Documents

File	What it does
`service.py`	Runs a brand-document job from start to finish. Also stores templates and results.
`reference.py`	**Learns a template** from a reference PDF: page types, colours, fonts, layout, logos and shapes.
`model.py`	The description of a learned template.
`content.py`	Reads your document into sections, paragraphs, lists, tables and the conclusion.
`fit.py`	**Fits the length** to the reference: shortens or lengthens with the local AI, checks every number and trims to size.
`imagegen.py`	Writes picture descriptions from the content, creates the pictures and colour-matches them to the brand.
`imagegen_worker.py`	The separate helper program that runs the picture AI (so its memory is freed afterwards).
`compose.py`	**Lays out the PDF pages** like the reference: cover, cards, columns, image pages, conclusion and back cover.
`fonts.py`	Uses the brand's own font if installed, or the brand's fall-back font.

3.5 `frontend/` — the screens you see (React)

File / folder	What it is
`index.html`	The empty page the app is loaded into.
`package.json`, `package-lock.json`	The list of building blocks for the screens.
`vite.config.ts`, `tsconfig.json`	Settings for building the screens.
`public/favicon.svg`	The small icon in the browser tab.
`dist/`	The **ready-built screens** (so computers without Node.js work too).
`src/main.tsx`	Starts the screens.
`src/App.tsx`	Decides which page shows for which address (the "map" of the app).
`src/index.css`	Shared look (fonts, colours).
`src/layouts/AppLayout.tsx`	The frame around every page: left menu and page header.
`src/services/api.ts`	Sends every request from the screens to the engine.

File / folder	What it is
`src/types/index.ts`	Descriptions of the data the screens receive.
`src/hooks/useAsync.ts`	Helper that loads data and shows progress.
`src/utils/format.ts`	Helpers for dates and sizes ("2 minutes ago").

Pages (`src/pages/`)

File	The screen
`Welcome.tsx`	First-run welcome and setup wizard.
`Dashboard.tsx`	Home: recent projects and system status.
`NewProject.tsx`	Create a project: sources, template, instruction, outputs.
`Projects.tsx`	The list of projects.
`ProjectDetail.tsx`	One project: presentation, images, video, sources, plan, files.
`ResearchPapers.tsx`	Convert a paper: upload, choose a format, list of conversions.
`PaperDetail.tsx`	One converted paper: preview, structure, references, report, downloads.
`BrandDocuments.tsx`	Brand Documents: templates, upload, length, options, list.
`BrandDocDetail.tsx`	One branded PDF: preview, images, check, create again.
`Templates.tsx`	Presentation templates.
`MediaLibrary.tsx`	The Media Library.
`Brands.tsx`	The list of brands.
`BrandEditor.tsx`	Edit one brand (colours, fonts, logos, references).
`Settings.tsx`	Settings and system status.

Building blocks (`src/components/`)

File	What it is
`ui.tsx`	Buttons, cards, badges, alerts, dialogs and tabs used everywhere.
`FileDrop.tsx`	The "drop files here" box.
`JobProgress.tsx`	The progress bar for running jobs.
`OutputPanels.tsx`	Result panels for presentations and images.
`Presenter.tsx`	Full-screen presentation mode.
`SlideEditor.tsx`	Editing a single slide.
`TemplateGallery.tsx`	The gallery of design templates.

File	What it is
`MediaPicker.tsx`	Choosing pictures and clips from the Media Library.
`VideoEditor.tsx`	The video timeline editor.

3.6 `brands/zensar/` — the Zensar brand

File	What it holds
`brand.json`	Brand name and general facts.
`colors.json`	The Zensar colours (indigo, coral, …) from official sources.
`typography.json`	Fonts: Graphik, with Arial as the fall-back.
`components.json`, `grid.json`, `layouts.json`, `shapes.json`	Rules for components, the layout grid, slide layouts and shapes.
`media.json`	How images are treated in the brand.
`social.json`	The "Experience" design language measured from Zensar's LinkedIn posts.
`video.json`, `voice.json`	Video and narration preferences.
`assets/logo/*`	The Zensar logo (white and dark, PNG and SVG).
`references/*`	Official brand reference images.

Put licensed **Graphik** font files in `brands/zensar/assets/fonts/` to use the exact typeface. Licensed fonts are never uploaded to GitHub.

3.7 `samples/` — examples to try

File	What it is
`InvoiceFlow_Showcase.docx`	A sample (fictional) document that uses every feature.
`demo_invoiceflow.mp4`	A sample product demo video.
`architecture_diagram.png`, `brand_backdrop.png`	Sample images.
`background_music.mp3`	Sample background music.
`narration_script.md`	A sample narration script.
`make_showcase.py`, `run_showcase.py`	Rebuild the samples / create the showcase project automatically.
`papers/sample_ieee_two_column.pdf`	A sample research paper (PDF) to convert.

File	What it is
`papers/sample_author_year.docx`	A sample research paper (Word).
`papers/sample_latex_project.zip`	A sample LaTeX paper project.
`papers/sample_raw_text.txt`	A sample paper as plain text.
`papers/make_sample_papers.py`	Rebuilds the sample papers.
`README.md` , `papers/README.md`	Explain the samples.

3.8 `tests/` — automatic checks

127 automatic checks that prove the app works. Developers run them with `python -m pytest` .

File	What it checks
`conftest.py`	Prepares a throw-away data folder for the tests.
`test_ingestion.py`	Reading every file type.
`test_planning.py`	Instruction understanding and slide planning.
`test_relevance_and_contract.py`	Choosing relevant facts and following the instruction.
`test_brand_and_rendering.py`	Brand rules and drawing slides.
`test_design_language.py`	The Zensar design language.
`test_media_features.py`	The Media Library and image features.
`test_video.py`	Video building and editing.
`test_api_and_privacy.py`	The API, and that nothing goes online.
`test_papers.py`	Research Papers: reading, references, all formats.
`test_branddocs.py`	Brand Documents: templates, length fitting, tables, layout.
`fixtures/*`	Generated sample files used by the tests.

3.9 Folders created while the app runs (on your computer only)

Folder	What is in it
`.venv/`	The app's private Python with its building blocks.
`data/`	Your uploads, projects, papers, brand templates and documents, and the database.
`outputs/` , `projects/`	Generated presentations, images and videos.
`models/`	The downloaded AI models.
`logs/`	Setup and server logs (useful when something fails).

Folder	What is in it
`offline/`	The offline bundle (only on networks that block pypi.org).

Frequently asked questions

Do my documents go to the internet? No. A built-in guard blocks all outside connections. Online image search is the only optional exception; it is off by default and never sends document text.

Why is creating images slow? Without an NVIDIA graphics card each picture takes 1–2 minutes. With one, it takes a few seconds.

The text says "Arial in place of Graphik"? Graphik is a licensed font. Copy its files into `brands/zensar/assets/fonts/` to use it.

Something went wrong. Look in the `logs` folder, then run `start.bat` again. The automatic tests (`python -m pytest`) show whether the installation is healthy.